

CSE 574 - Computational Complexity

Zhengjie Min

September 1, 2026

Abstract

This is a template for TCS (and other kind of) lecture notes.

Contents

I	Introduction & Background Review	2
1	The Beginning	3
1.1	Definition of Computational Theory	3
1.2	What is Complexity About?	4
2	Time and Non-deterministic Time	5
2.1	Languages and Problems	5
2.2	Classes of Complexity	5
A	Algebra	7
B	Analysis	8

Part I

Introduction & Background Review

Chapter 1

The Beginning

The start of computational theory can be traced back to the 1930s, when mathematicians and logicians were trying to formalize the notion of computation and algorithms. The Church-Turing thesis, proposed independently by Alonzo Church and Alan Turing, laid the foundation for the study of computational complexity.

Theorem 1.0.1 (Church-Turing Thesis). Every effectively calculable function is computable by a Turing machine.

Then mathematicians and computer scientists began to explore the limits of computation, with such an open thesis:

Problem 1.0.1 (Extended (Quantum) Church-Turing Thesis). Every physically realizable model of computation can be simulated by a (quantum) Turing machine efficiently.

1.1 Definition of Computational Theory

Remark 1.1.1. Computational theory deals with how much resources we need to solve a computational problem.

Resources can be many things. For example:

- Time
- Memory / Space
- Samples Complexity (how many samples we need to distinguish between distributions)
- Communication Rounds
- Oracle Calls
- Randomness
- Power
- Quantum Entanglement
- Number of Parallel Workers
- ...

1.2 What is Complexity About?

In CS Theory, people do research in algorithms and complexity. People might lean on one side or the other, but they are closely related. In a word:

Remark 1.2.1. Algorithms are about the upper bound of a problem, while complexity is about the lower bound of a problem.

Chapter 2

Time and Non-deterministic Time

2.1 Languages and Problems

In computational complexity, we talk about languages as how we think about computational problems.

Definition 2.1.1. A language L is a set of strings that $L \subseteq \Sigma^*$, where Σ is a finite alphabet (often $\{0, 1\}$ bits.)

Example 2.1.1. An algorithm $\text{PRIMES}(n)$ can decide whether a number n is prime or not. The language of this problem is:

$$\text{PRIMES} = \{\langle x \rangle : x \text{ is prime}\}.$$

$\langle x \rangle$ is the encoding of x in binary.

2.2 Classes of Complexity

Definition 2.2.1. $L \in \text{DTIME}(T(n))$ if there's a deterministic Turing machine M that decides L in time $O(T(n))$. $P := \bigcup_{c \geq 1} \text{DTIME}(n^c)$ is the class of problems that can be solved in polynomial time.

It's easy to see that $\text{PRIMES} \subseteq P$.

Appendix

Appendix A

Algebra

Theorem A.0.1. For any polynomial $A \in \mathbb{C}[X]$ of degree $n - 1$, any point set $\{p_0, \dots, p_{n-1}\}$ of size n , $\{(p_i, A(p_i))\}_{i=0}^{n-1}$ determines $A(x)$ and vice versa.

Theorem A.0.2 (Fundamental Theorem of Algebra). Every non-constant single-variable polynomial with complex coefficients has at least one complex root.

In math terms, for $A(x) = \sum_{i=0}^{n-1} a_i x^i$, either $\exists x \in \mathbb{C}$ such that $A(x) = 0$, or $a_0 = a_1 = \dots = a_{n-1} = 0$.

Appendix B

Analysis

Theorem B.0.1 (Master Theorem (simplified)). Let the recurrence relation be:

$$T(n) = aT\left(\frac{n}{b}\right) + n^d$$

where $a \geq 1$ is the number of subproblems, $b > 1$ is the factor by which the problem size is reduced, and $d \geq 0$. The asymptotic bound of $T(n)$ is given by:

1. If $\log_b a > d$, then $T(n) = O(n^{\log_b a})$.
2. If $\log_b a = d$, then $T(n) = O(n^d \log n)$.
3. If $\log_b a < d$, then $T(n) = O(n^d)$.

Theorem B.0.2 (Master Theorem). Let $a \geq 1$ and $b > 1$ be constants, let $f(n)$ be a function, and let $T(n)$ be defined on the nonnegative integers by the recurrence:

$$T(n) = aT(n/b) + f(n)$$

Then $T(n)$ has the following asymptotic bounds:

- **Case 1:** If $f(n) = O(n^{\log_b a - \epsilon})$ for some constant $\epsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$.
- **Case 2:** If $f(n) = \Theta(n^{\log_b a} \log^k n)$ for some constant $k \geq 0$, then $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$.
- **Case 3:** If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some constant $\epsilon > 0$, and if $af(n/b) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n (Regularity Condition), then $T(n) = \Theta(f(n))$.